

Software para Sistemas Ubíquos

Do protótipo individual à integração
Middleware, plataformas e Marco 2

Otávio Calaça Xavier

Hoje fechamos um ciclo e abrimos outro

1 · DEMONSTRAR

Alguns estudantes executam seus protótipos e tornam visíveis decisões do Marco 1.

2 · INTEGRAR NO GRUPO

Os integrantes comparam comportamentos e selecionam o que será aproveitado coletivamente.

3 · INICIAR O MARCO 2

A teoria orienta o planejamento. A conclusão e a verificação ocorrerão em 18/09.

Hoje há planejamento do Marco 2 — não há entrega nem apresentação final.

Objetivos de aprendizagem

- Analisar decisões técnicas demonstradas nos protótipos do Marco 1.
- Comparar soluções individuais a partir de comportamento executável.
- Distinguir protocolo, middleware, plataforma e framework.
- Comparar MQTT e HTTP e reconhecer os possíveis papéis de uma aplicação Android.
- Iniciar um plano de integração coerente com os requisitos do projeto.

Agenda — Aula 5

- | | |
|----------------------------|-----------------------------|
| 1 Abertura e objetivos | 4 Comparação nos grupos |
| 2 Demonstrações do Marco 1 | 5 Fundamentos de integração |
| 3 Síntese coletiva | 6 Início do Marco 2 |

Demonstração → comparação → teoria → planejamento.

Apresentações do Marco 1: observem decisões

CONTEXTO

Que fenômeno e situação o protótipo representa?

ESTADO E REGRA

Que memória, tempo ou qualidade impede uma decisão ingênua?

FALHA

Como o comportamento muda diante de dado inválido, silêncio ou oscilação?

LIMITES

O que a simulação demonstra — e o que permanece sem validação?

Procure uma decisão que poderia fortalecer o protótipo do seu grupo.

Roteiro de demonstração — até cinco minutos

- 1 Apresente o recorte e a responsabilidade assumida.
- 2 Execute uma entrada, uma transição de estado e uma resposta.
- 3 Mostre o teste mais informativo — não apenas o caso feliz.
- 4 Explique por que o programa decidiu dessa forma.
- 5 Declare uma limitação e responda à pergunta da turma.

Pergunta possível: o que acontece se esta entrada mudar agora?

Síntese das demonstrações

DECISÕES REUTILIZÁVEIS

- validação e faixa
- estado e tempo
- histerese ou debounce
- falha explícita
- logs explicáveis

LIMITAÇÕES EXPOSTAS

- contexto insuficiente
- sensor silencioso
- dependência local
- contrato implícito
- ausência de comunicação

Que problema só aparece quando outro componente precisa participar?

INTEGRAÇÃO

Comparar e integrar no grupo

Do comportamento individual às decisões coletivas.

Agora o grupo compara executando

- 1 Executem pelo menos dois protótipos — não comparem apenas relatórios.
- 2 Escolham uma decisão de cada solução que mereça ser preservada.
- 3 Identifiquem incompatibilidades de evento, estado e atuação.
- 4 Seleccionem o primeiro fluxo que deverá atravessar dois componentes.
- 5 Distribuem responsabilidades para o planejamento do Marco 2.

Tempo: 35 minutos.

Compare comportamento — não quantidade de componentes

ENTRADA

Como reconhece leitura inválida, instável ou ausente?

ESTADO

Que informação anterior modifica a decisão atual?

DECISÃO

Qual regra é mais explicável e segura?

FALHA

Qual solução degrada de maneira mais adequada?

OBSERVAÇÃO

Os logs permitem reconstruir a decisão?

A melhor escolha depende do risco e do requisito — não do tamanho do código.

Selecionar também significa adaptar e descartar

PRESERVAR

Comportamento coerente, executável e necessário ao fluxo coletivo.

ADAPTAR

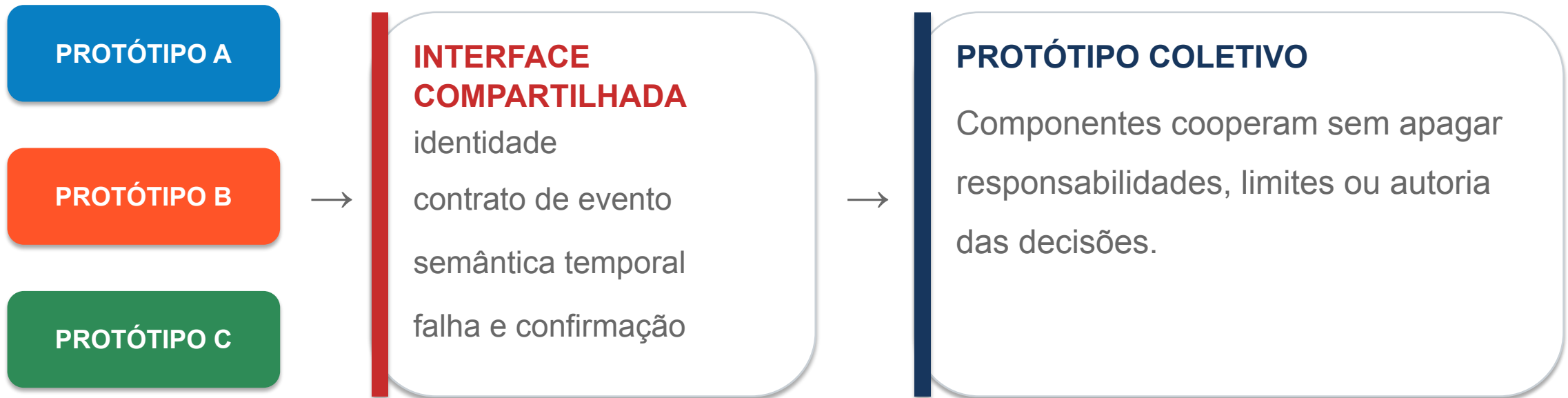
Boa decisão, mas com contrato, estado ou interface incompatível.

DESCARTAR

Complexidade sem requisito, duplicação ou risco não tratado.

Justifique escolhas a partir do comportamento observado.

Do protótipo individual ao protótipo coletivo



Integrar não é copiar tudo para o mesmo programa.

COMPREENSÃO

Compreender a integração

Conceitos, camadas e tecnologias que orientam o Marco 2.

Integração cria problemas que não existiam localmente

IDENTIDADE E CONTRATO

Quem produziu? Qual versão? Como interpretar?

TEMPO E ORDEM

Chegou tarde, repetido ou fora de ordem?

FALHA E RECUPERAÇÃO

O envio falhou ou apenas a resposta se perdeu?

SEGURANÇA E PRIVACIDADE

Quem pode observar, publicar ou comandar?

A comunicação transforma decisões locais em compromissos distribuídos.

Quatro termos que não são sinônimos

PROTOCOLO

Regras de comunicação entre participantes.

Ex.: HTTP, MQTT.

MIDDLEWARE

Media interação e oferece capacidades recorrentes.

Ex.: broker, autenticação, descoberta.

PLATAFORMA

Ambiente integrado para executar, operar e observar soluções.

Ex.: serviço IoT gerenciado.

FRAMEWORK

Estrutura e abstrações que orientam a organização do código.

Ex.: Jetpack Compose, Spring.

Uma solução pode combinar os quatro — cada um responde a uma função diferente.

Classifique o elemento concreto — não apenas a tecnologia

MQTT

O protocolo define pacotes e semântica. O broker executável exerce mediação entre clientes.

HTTP

O protocolo define requisições e respostas. Uma API define recursos e operações do domínio.

ANDROID

É uma plataforma de execução. O aplicativo usa SDKs, bibliotecas e protocolos para integrar.

KAFKA / gRPC

Kafka oferece uma plataforma de eventos; gRPC fornece um framework de chamadas remotas tipadas.

Pergunte sempre: qual parte está sendo classificada e qual responsabilidade ela assume?

A integração atravessa camadas técnicas diferentes

APLICAÇÃO E DOMÍNIO

evento, comando, regra, recurso e consequência

FRAMEWORKS E BIBLIOTECAS

SDK Android, cliente MQTT, Retrofit, gRPC

MIDDLEWARE E SERVIÇOS

broker, API gateway, autenticação, filas e observabilidade

PROTOCOLOS DE APLICAÇÃO

MQTT, HTTP e protocolo gRPC sobre HTTP/2

TRANSPORTE E REDE

TCP, TLS, IP, Wi-Fi, rede móvel e Ethernet

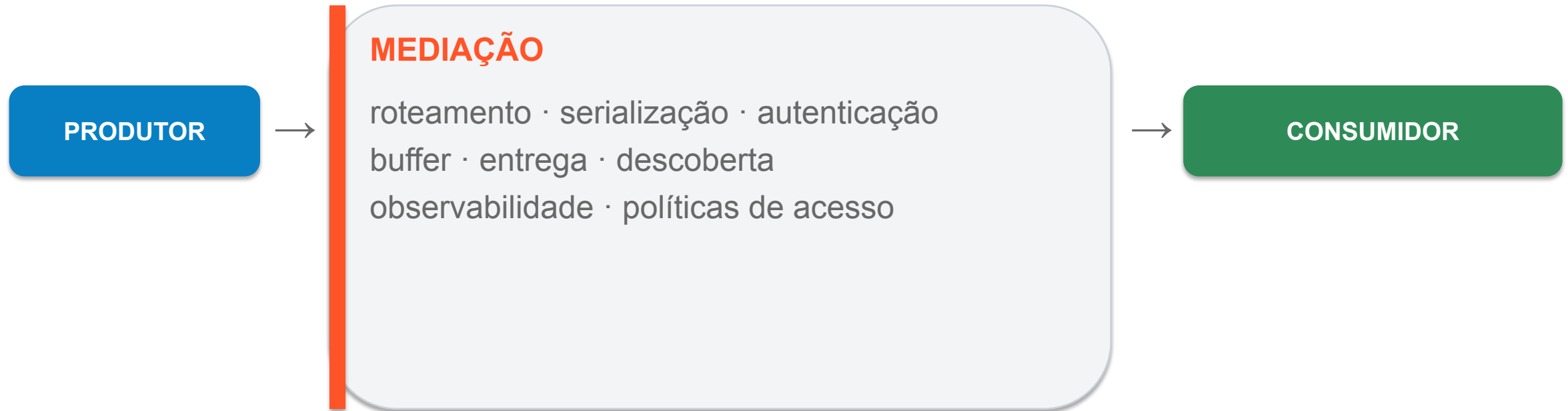
Escolher MQTT ou HTTP não resolve sozinho contrato, segurança, estado ou implantação.

Uma tecnologia pode participar de mais de uma categoria

ELEMENTO	O QUE É	PAPEL NA SOLUÇÃO
Broker MQTT	software de mediação	roteia publicações, mantém sessões e aplica políticas
API HTTP	interface da aplicação	expõe recursos e operações; não é o próprio HTTP
Android	plataforma	executa o app e fornece ciclo de vida, permissões e conectividade
Kafka	plataforma de streaming	armazena e distribui eventos duráveis para consumidores
gRPC	framework RPC	gera clientes/servidores a partir de contratos tipados

As categorias explicam responsabilidades; não servem apenas para rotular produtos.

Middleware concentra responsabilidades recorrentes



Plataformas podem oferecer essas capacidades como serviços; frameworks ajudam a incorporá-las ao código.

Middleware não elimina decisões de arquitetura — torna algumas delas explícitas e operáveis.

O contrato agora atravessa uma fronteira

```
{  
  "schemaVersion": 1,  
  "eventId": "evt-1842",  
  "correlationId": "cmd-309",  
  "eventType": "room.state.changed",  
  "deviceId": "esp32-room-01",  
  "eventTime": "2026-09-11T19:14:32-03:00",  
  "sequence": 1842,  
  "state": "ATTENTION"  
}
```

Versão

permite evoluir o formato

ID do evento

permite reconhecer repetição

Correlação

conecta comando, evento e resposta

Tempo e sequência

apoiam validade e ordem

Telemetria e comando exigem compromissos diferentes

TELEMETRIA · O QUE FOI OBSERVADO

- pode ser frequente
- pode tolerar alguma perda
- possui validade temporal
- alimenta estado e histórico
- atraso reduz sua utilidade

COMANDO · O QUE DEVE ACONTECER

- produz consequência
- pode exigir autorização
- repetição pode ser perigosa
- confirmação precisa de significado
- expiração pode impedir atuação tardia

Receber um pacote não comprova que a ação foi executada.

Acoplamento possui mais de uma dimensão

ESPAÇO

O produtor precisa conhecer o endereço do consumidor?

TEMPO

Os dois precisam estar disponíveis ao mesmo tempo?

CONTRATO

Uma mudança de campo quebra os participantes?

FALHA

A indisponibilidade de um componente paralisa os demais?

Desacoplar um aspecto pode acrescentar estado, infraestrutura e operação.

Entrega não é o mesmo que processamento



CONFIRMAÇÃO DE TRANSPORTE

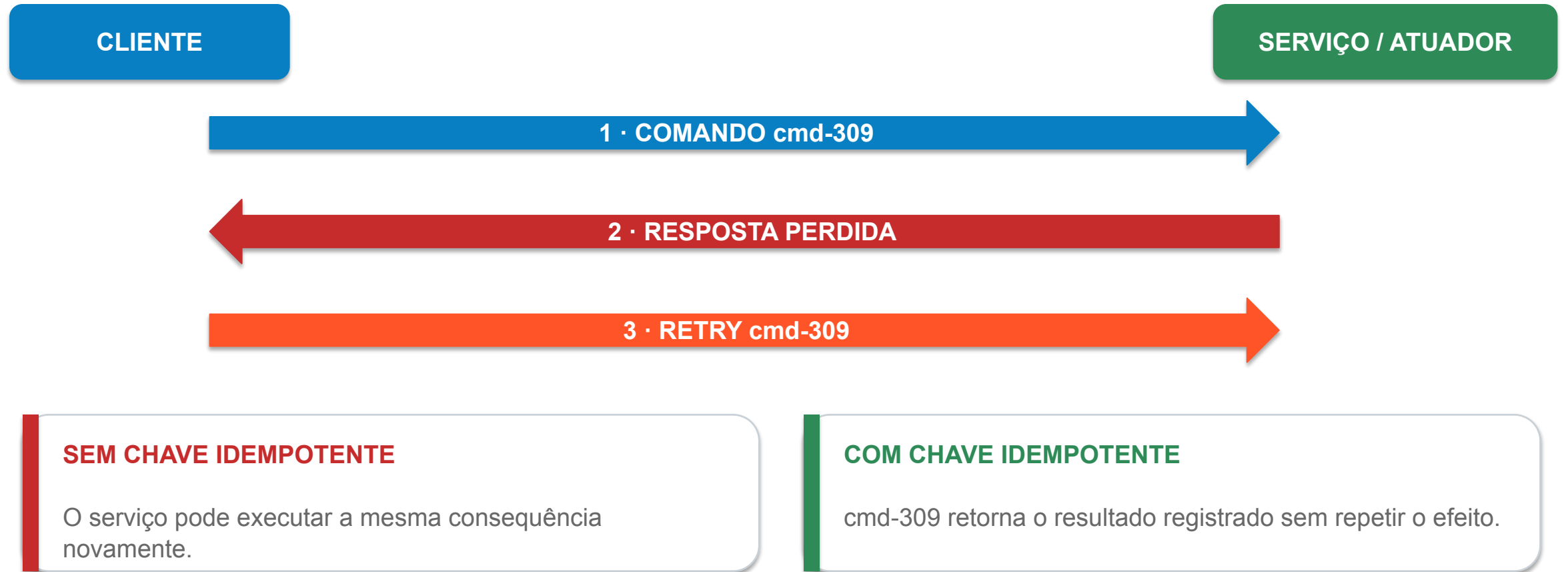
Indica avanço no protocolo — não o resultado da regra de negócio.

CONFIRMAÇÃO DE NEGÓCIO

Declara aceitação, rejeição, execução ou estado atual.

Defina qual confirmação o seu requisito realmente necessita.

Repetir uma tentativa pode repetir o efeito



Quando a rede desaparece, o estado permanece



ARMAZENAR

Quais eventos merecem sobreviver à desconexão?

EXPIRAR

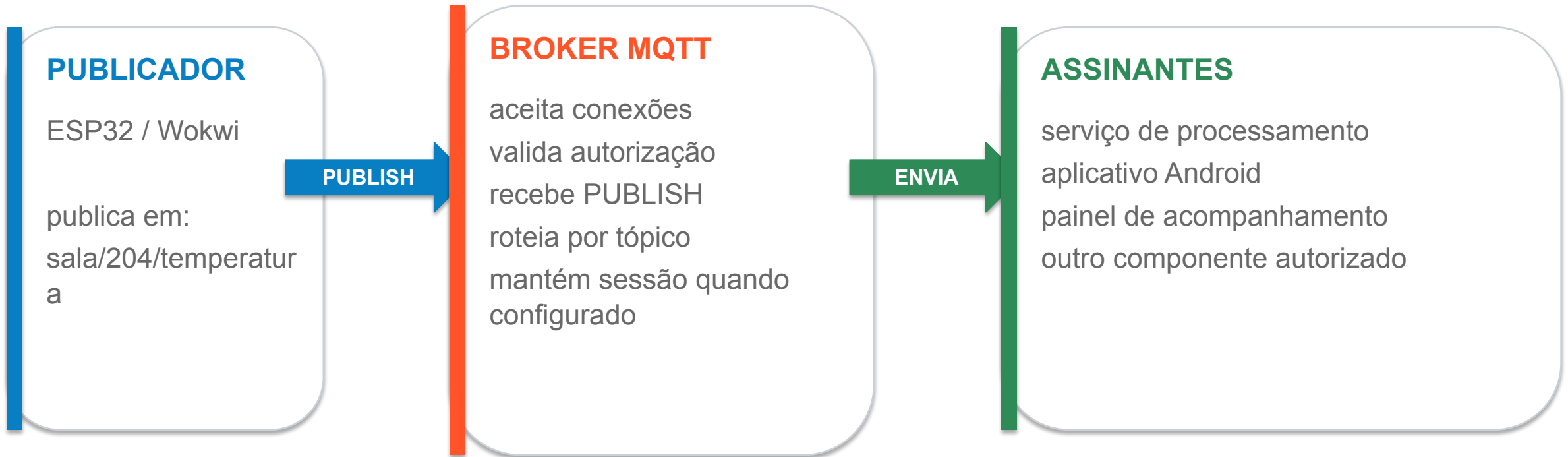
Quando o evento deixa de orientar o presente?

SINCRONIZAR

Como evitar repetição e inversão de ordem?

Modo offline é uma política de dados e decisão — não apenas uma tela de erro.

MQTT separa produtores e consumidores por um broker



O protocolo é MQTT; o broker exerce a função de middleware.

Uma sessão MQTT começa antes da primeira publicação

CONNECT

cliente propõe identidade, sessão, autenticação e keep alive

CONNACK

broker aceita ou rejeita a conexão

SUBSCRIBE

assinante registra filtros de tópico e QoS

PUBLISH

publicador envia tópico, propriedades e payload

ACK / KEEP ALIVE

confirmações e PING mantêm o estado do protocolo

MQTT mantém estado de conexão; o seu domínio ainda precisa manter estado de negócio.

INF

INSTITUTO DE
INFORMÁTICA



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS



QoS descreve uma troca entre cliente e broker

QoS 0 · NO MÁXIMO UMA VEZ

Um PUBLISH é enviado sem confirmação do receptor.

Menor custo; perda é possível.

QoS 1 · AO MENOS UMA VEZ

O receptor confirma com PUBACK. Retransmissão pode produzir duplicatas.

QoS 2 · EXATAMENTE UMA VEZ

Uma sequência adicional evita entrega duplicada naquele enlace MQTT, com maior custo.

QoS não garante que o consumidor validou, decidiu ou atuou corretamente.

Em MQTT, quatro escolhas mudam o comportamento

TÓPICO

Expressa domínio e finalidade; orienta quem receberá a publicação.

QoS

0: melhor esforço · 1: ao menos uma entrega · 2: entrega protocolar única.

RETAINED MESSAGE

O broker preserva a última publicação retida para novos assinantes.

WILL MESSAGE

O broker pode publicar uma mensagem se a conexão terminar sem o encerramento esperado.

QoS trata entrega MQTT; não garante que a regra de negócio foi executada.

Um tópico é parte da interface

```
campus/bloco-a/sala-204/telemetry/temperature
```

```
campus/bloco-a/sala-204/state/occupancy
```

```
campus/bloco-a/sala-204/command/hvac
```

EVITE

nomes frágeis como teste1, dados ou sensor.

DECIDA

hierarquia · autorização
telemetria × comando
estabilidade do nome

Mudar um tópico publicado pode quebrar assinantes que você não conhece.

Em HTTP, protocolo, API e servidor são elementos distintos

PROTOCOLO HTTP

Define métodos, URIs, cabeçalhos, conteúdo, status e semântica de requisição/resposta.

API

Contrato da aplicação: recursos, operações, representações, erros e regras de autorização.

SERVIDOR / FRAMEWORK

Implementa a API e o domínio; pode usar Spring, FastAPI, Express ou outra estrutura.

INFRAESTRUTURA INTERMEDIÁRIA

Proxy, API gateway, cache e balanceador podem mediar segurança, tráfego e observação.

Dizer “vamos usar HTTP” ainda não define a API nem a responsabilidade do serviço.

Uma troca HTTP carrega intenção e resultado

REQUISIÇÃO

POST /api/v1/rooms/204/commands HTTP/1.1

Content-Type: application/json

Idempotency-Key: cmd-309

```
{"action": "turnOnHvac", "expiresAt": "19:15:00-03:00"}
```

RESPOSTA

HTTP/1.1 202 Accepted

Content-Type: application/json

Location: /api/v1/commands/cmd-309

```
{"commandId": "cmd-309", "status": "accepted"}
```

202 Accepted confirma aceitação — não comprova que a atuação já terminou.

INF

INSTITUTO DE
INFORMÁTICA



UFG

UNIVERSIDADE
FEDERAL DE GOIÁS



HTTP organiza uma interação explícita de requisição e resposta



O método declara intenção; a aplicação ainda define o significado do recurso.

Status e idempotência orientam a recuperação em HTTP

2xx · SUCESSO

A requisição foi recebida e tratada segundo a semântica do status.

4xx · PROBLEMA NO PEDIDO

Repetir sem corrigir autenticação, contrato ou estado tende a falhar novamente.

5xx · FALHA NO SERVIÇO

Uma nova tentativa pode ser útil, mas exige limite, espera e idempotência.

GET, PUT e DELETE possuem semântica idempotente no HTTP. POST não se torna seguro para repetição apenas por usar HTTP.

Falha antes da resposta não informa se o servidor aplicou ou não o pedido.

Android é um participante — não um protocolo

FONTE DE CONTEXTO

Localização, movimento, interação e estado do dispositivo podem produzir eventos.

INTERFACE HUMANA

Apresenta estado, solicita confirmação e permite comando explícito.

CLIENTE / MEDIADOR

Pode consumir API HTTP, assinar MQTT, usar BLE e manter dados locais para operação offline.

Separe interface, dados e fontes externas para tornar comportamento testável.

INF

INSTITUTO DE
INFORMÁTICA



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS



No Android, a integração pertence à camada de dados



Offline exige uma fonte local e uma política de sincronização — não apenas retry.

gRPC e Kafka resolvem problemas diferentes

gRPC · CHAMADA REMOTA TIPADA

- serviço e mensagens definidos em arquivos .proto
- geração de clientes e servidores
- chamadas unary e streaming
- normalmente transportado sobre HTTP/2
- forte contrato entre participantes

KAFKA · STREAM DE EVENTOS DURÁVEL

- eventos persistidos em tópicos particionados
- consumidores controlam seu progresso
- replay e múltiplos consumidores
- coordenação e operação do cluster
- adequado quando retenção e escala são requisitos

Nenhum dos dois é uma evolução automática de MQTT ou HTTP.

Não existe um mecanismo universalmente melhor

MECANISMO	FORÇA	ATENÇÃO
MQTT	distribuição de eventos e desacoplamento	broker, tópicos, sessão e duplicação
HTTP	interação explícita e ampla interoperabilidade	disponibilidade, retry e semântica do recurso
gRPC	interfaces tipadas entre serviços	compatibilidade, infraestrutura e clientes
Kafka	stream durável, replay e múltiplos consumidores	operação e escala podem exceder a necessidade

Comece pelo requisito; use o menor mecanismo que o atenda.

Três composições coerentes — para necessidades diferentes

EVENTOS DESACOPLADOS

ESP32 / Wokwi

↓ publica

MQTT broker

↓ distribui

serviço + Android

API EXPLÍCITA

ESP32 / cliente

↓ requisição

API HTTP

↓ resposta

banco + painel

MÓVEL COMO MEDIADOR

sensor próximo

↓ BLE

Android

↓ MQTT ou HTTP

serviço remoto

Misturar mecanismos é válido quando cada fronteira possui uma necessidade distinta.

INF

INSTITUTO DE
INFORMÁTICA



UFG

UNIVERSIDADE
FEDERAL DE GOIÁS



Escolha a partir dos requisitos do seu projeto

MÚLTIPLOS CONSUMIDORES?

Considere publicação e assinatura.

CONSULTA OU AÇÃO EXPLÍCITA?

Considere uma API baseada em recursos.

PRECISA OPERAR SEM REDE?

Defina estado local, fila, expiração e sincronização.

COMANDO PRODUZ CONSEQUÊNCIA?

Inclua autorização, identidade, validade e confirmação.

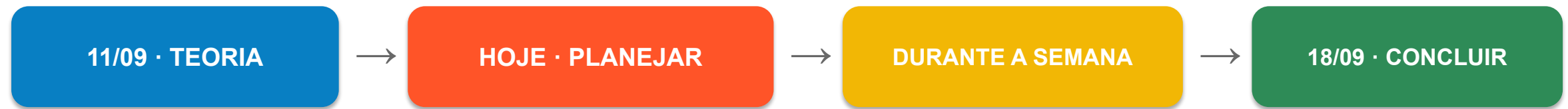
A justificativa deve ligar mecanismo, risco e comportamento esperado.

PRÁTICA

Iniciar o Marco 2

Planejar hoje; construir e verificar em 18/09.

O Marco 2 começa hoje e termina na próxima semana



HOJE

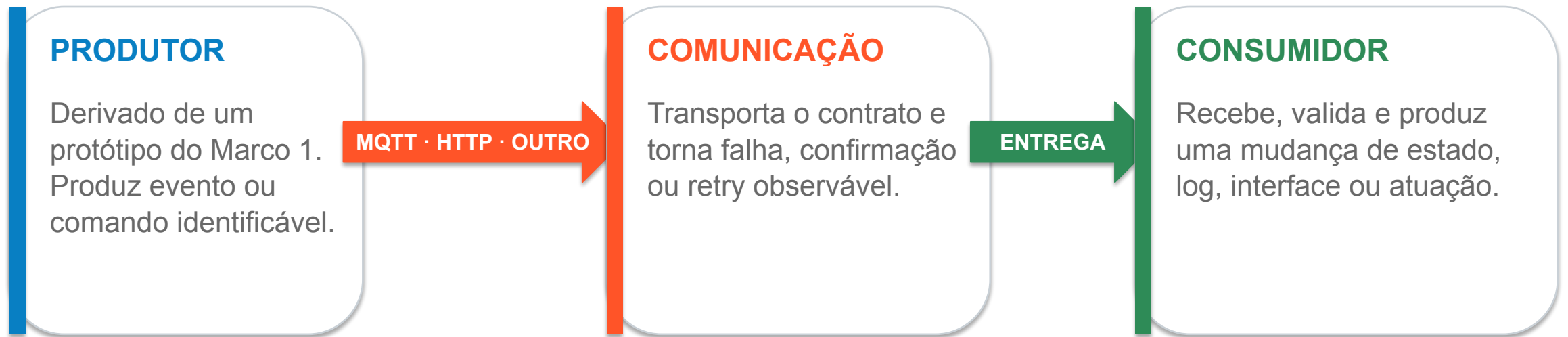
Definir fluxo, contrato inicial, mecanismo e responsabilidades.

PRÓXIMO ENCONTRO

Executar, verificar, explicar e ajustar a integração.

Não há entrega nem apresentação do Marco 2 hoje.

Desafio do Marco 2 — produzir uma cooperação observável



O fluxo deve reutilizar decisões relevantes dos protótipos individuais e tratar pelo menos uma condição de falha.

O objetivo não é integrar o projeto inteiro — é validar uma fronteira real entre componentes.

O grupo precisa sair hoje com cinco decisões

- 1 Qual componente produzirá o primeiro evento ou comando?
- 2 Qual componente deverá consumi-lo e produzir efeito observável?
- 3 Qual é o contrato mínimo e como será versionado?
- 4 Qual mecanismo atende ao requisito prioritário?
- 5 Quem fará cada parte e qual primeiro passo poderá ser executado?

Decisões podem mudar quando o grupo encontrar evidência nova.

Entrega esperada do Marco 2 — 18/09

NO REPOSITÓRIO DO GRUPO

- código do produtor
- código do consumidor
- configuração necessária
- contrato ou payload utilizado
- instrução mínima para executar

NA DEMONSTRAÇÃO

- troca executada ao vivo
- evento com identificação única
- efeito observável no consumidor
- falha ou retry exercitado
- versão igual à registrada

NA VERIFICAÇÃO ORAL

- explicar o contrato
- justificar o mecanismo
- relacionar risco e comportamento
- localizar estado e responsabilidade
- realizar pequena alteração

Não será exigido relatório discursivo nem apresentação de slides pelo grupo.

O que não é exigido hoje

NÃO É ENTREGA

Nenhum grupo precisa concluir ou enviar o Marco 2 nesta aula.

NÃO É ARQUITETURA FINAL

O plano inicial pode ser corrigido durante a implementação.

NÃO É RELATÓRIO EXTENSO

A evidência principal será execução, código e explicação.

Planejar hoje significa reduzir a próxima incerteza executável.

O que será verificado em 18/09

TROCA EXECUTÁVEL

Um produtor envia; um consumidor recebe e torna o efeito observável.

CÓDIGO E CONFIGURAÇÃO

A versão demonstrada está registrada no repositório do grupo.

EXPLICAÇÃO ORAL

Os integrantes relacionam contrato, mecanismo, falha e requisito.

PEQUENA ALTERAÇÃO

O grupo prevê e executa uma mudança solicitada durante a verificação.

Uma imagem ou um texto isolado não comprova a integração.

Uso de IA deve produzir compreensão verificável

PODE APOIAR

Explorar alternativas, explicar erros, revisar código e propor testes.

PRECISA SER AVALIADA

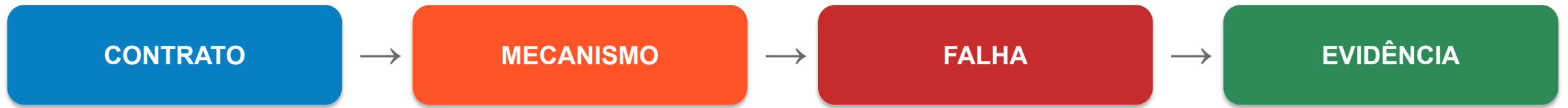
Sugestões podem errar contratos, bibliotecas, segurança e condições de falha.

NÃO SUBSTITUI

Execução, autoria das decisões, explicação e capacidade de modificar.

Todo integrante responde pelo comportamento que o grupo demonstra.

Síntese — do componente à cooperação



- MQTT, HTTP e Android exercem papéis diferentes na arquitetura.
- A escolha deve responder a requisitos e consequências observáveis.
- O plano começa hoje; a integração será concluída e verificada em 18/09.

Próximo passo: abrir o repositório do grupo e definir as cinco decisões iniciais.

Referências e apoio

OASIS. MQTT Version 5.0 — OASIS Standard.

<https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

IETF. RFC 9110 — HTTP Semantics.

<https://www.rfc-editor.org/rfc/rfc9110.html>

Android Developers. Recommendations for Android architecture.

<https://developer.android.com/topic/architecture/recommendations>

Android Developers. Data layer and offline-first guidance.

<https://developer.android.com/topic/architecture/data-layer>

Apache Kafka. Introduction and event streaming concepts.

<https://kafka.apache.org/documentation/>

gRPC. Introduction and core concepts.

<https://grpc.io/docs/what-is-grpc/introduction/>

Material da disciplina: Aulas 01–04 e Atividades 01–03.